Chapter 4.



SCHOOL OF COMPUTING & DATA SCIENCE
The University of Hong Kong

Android Layout

2026-2027
COMP3330 Interactive Mobile Application Design and Programming
Dr. T.W. Chim (E-mail: twchim.cs@hku.hk)

Agenda

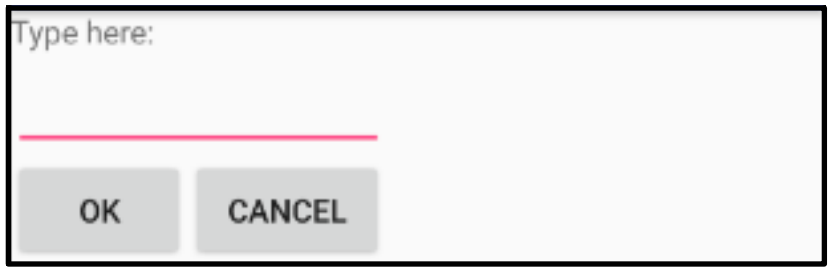
- Concept of View in Android
- Android Layout
- Android Dimension Issues



Concept of View in Android

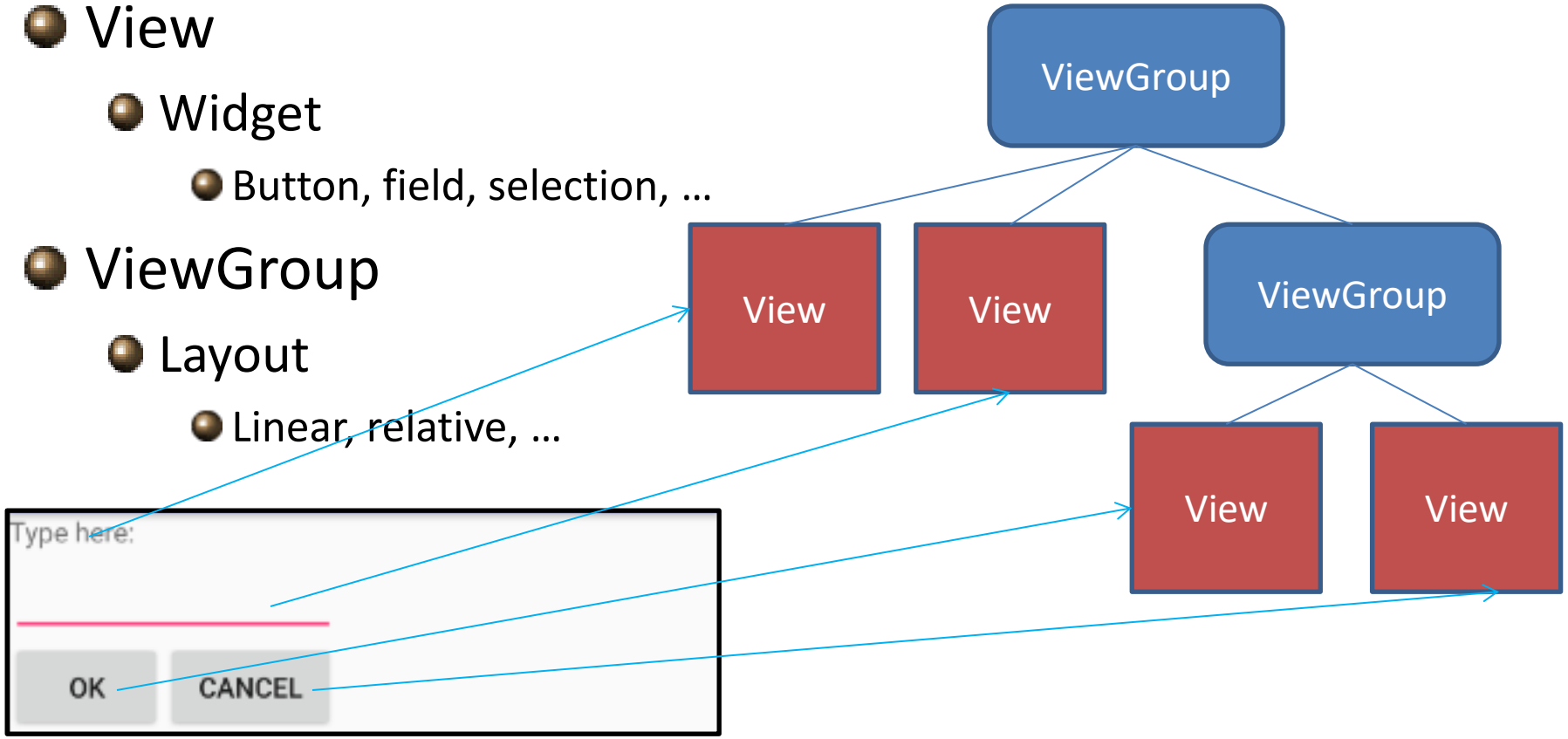
Views

- The visible parts of Android apps
- Basic building components for UI
- Each activity / app is composed of one or more views
  - Each view is an Android class
  - In charge of a rectangle area
  - Handles
    - Drawing
    - Measurement
    - User interactions (events)
      - Scroll, press, key, …
      - Focus change



View Hierarchy

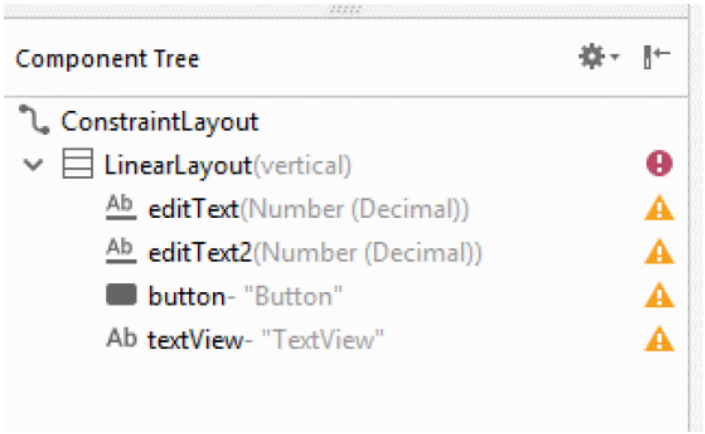
- View
  - Widget
    - Button, field, selection, …
- ViewGroup
  - Layout
    - Linear, relative, …

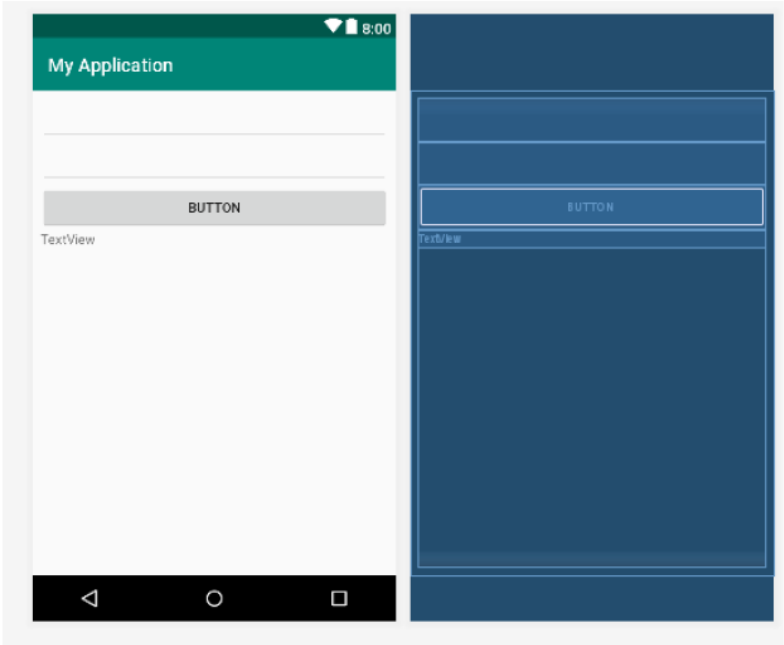


Drawing is done from the root.

View Hierarchy

- How about this?

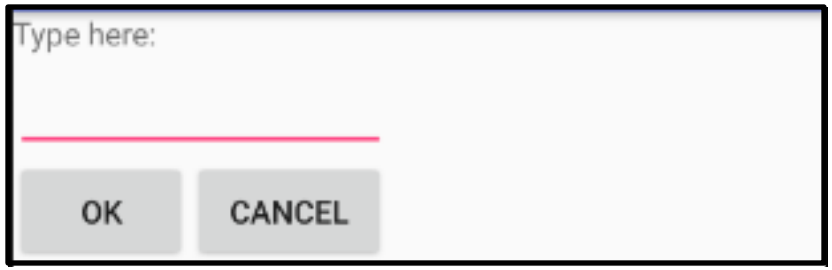




- ConstraintLayout: ViewGroup
- LinearLayout: ViewGroup
- editText: View
- editText2: View
- button: View
- textView: View

Properties of View

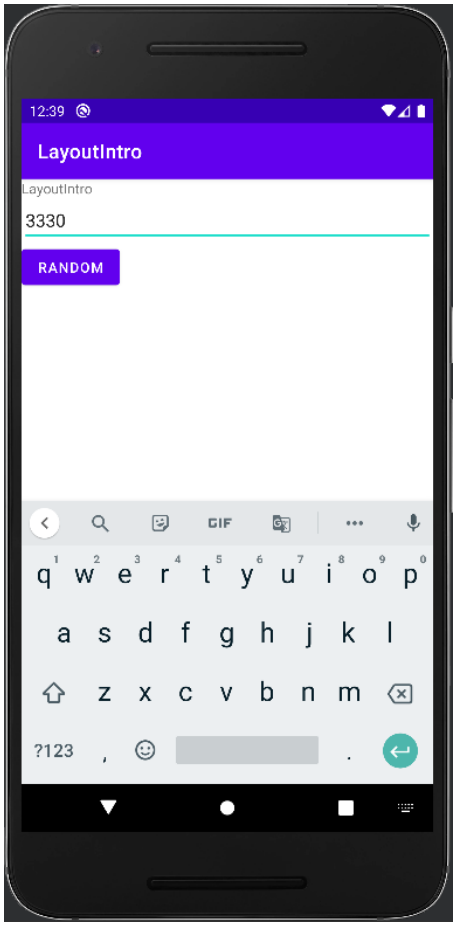
- Static appearance
  - Layout, widget
- Dynamic behavior
  - Event handling
- How to learn?
  - Look at examples
  - Have a rough idea / impression first, look up later
    - Breadth first traversal
    - Nonlinear



Example - Layout

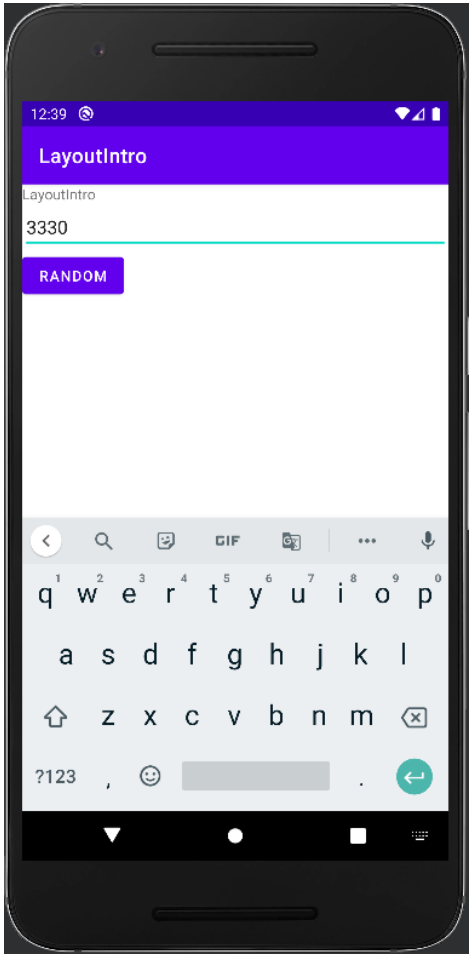
```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
        android:layout_width="fill_parent"
        android:layout_height="fill_parent"
        android:orientation="vertical" >
        <TextView
                android:layout_width="fill_parent"
                android:layout_height="wrap_content"
                android:text="@string/app_name" />
        <EditText
                android:id="@+id/editText1"
                android:layout_width="fill_parent"
                android:layout_height="wrap_content" >
        </EditText>
        <Button
                android:id="@+id/button1"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="RANDOM" />
</LinearLayout>
```

Note the 2 ways of closing tags.



Example – Java Code

"R": resources
"R.id": resource ID (corresponds to "@+id" in layout file)




```
public class MainActivity extends AppCompatActivity {
        private Random random;
        public void onCreate(Bundle savedInstanceState) {
                super.onCreate(savedInstanceState);
                setContentView(R.layout.activity_main);
                random = new Random();

                final EditText edit_text1 = (EditText) findViewById(R.id.editText1);
                final Button button1 = (Button) findViewById(R.id.button1);

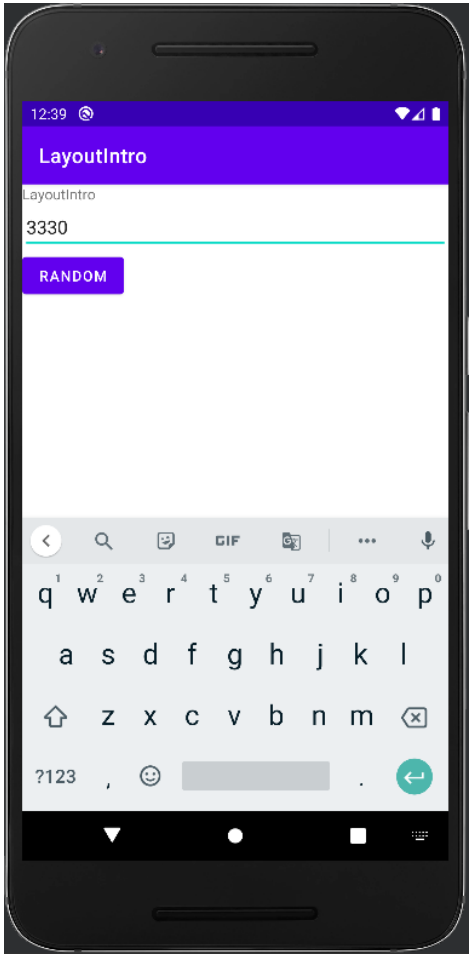
                Button.OnClickListener listener =  new Button.OnClickListener() {
                        public void onClick(View dummy) {
                                edit_text1.setText(String.valueOf(random.nextInt(100)));
                        }
                };

                button1.setOnClickListener(listener);
        }
}
```

"final" = always contain the same value

Example – Kotlin Code

“R”: resources
“R.id”: resource ID (corresponds to “@+id” in layout file)



"?" notation indicates that a data type can take a null value.

```
class MainActivity : AppCompatActivity() {
        private var random: Random? = null
        public override fun onCreate(savedInstanceState: Bundle?) {
                super.onCreate(savedInstanceState)
                setContentView(R.layout.activity_main)
                random = Random()

                val edit_text1 = findViewById<View>(R.id.editText1) as EditText
                val button1 = findViewById<View>(R.id.button1) as Button

                val listener = View.OnClickListener {
                        edit_text1.setText(random!!.nextInt(100).toString())
                }

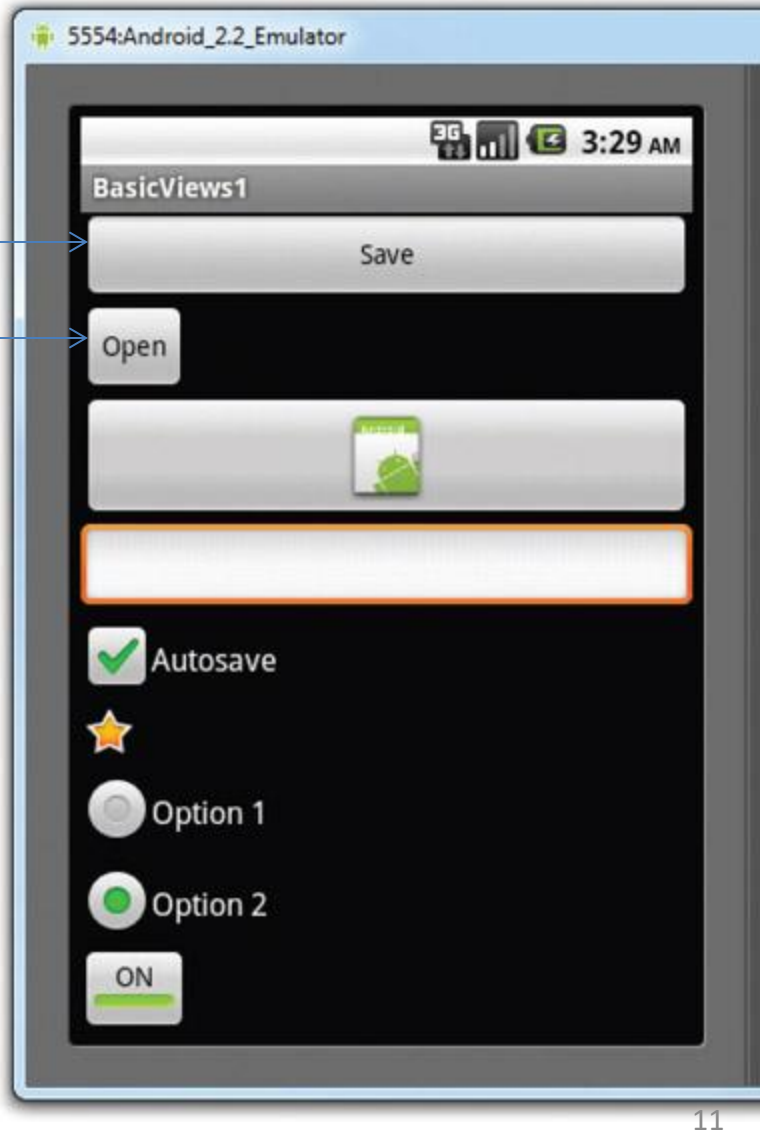
                button1.setOnClickListener(listener)

        }
}
```

“random” is non-null type and throws an exception if the value is null.

match_parent & wrap_content

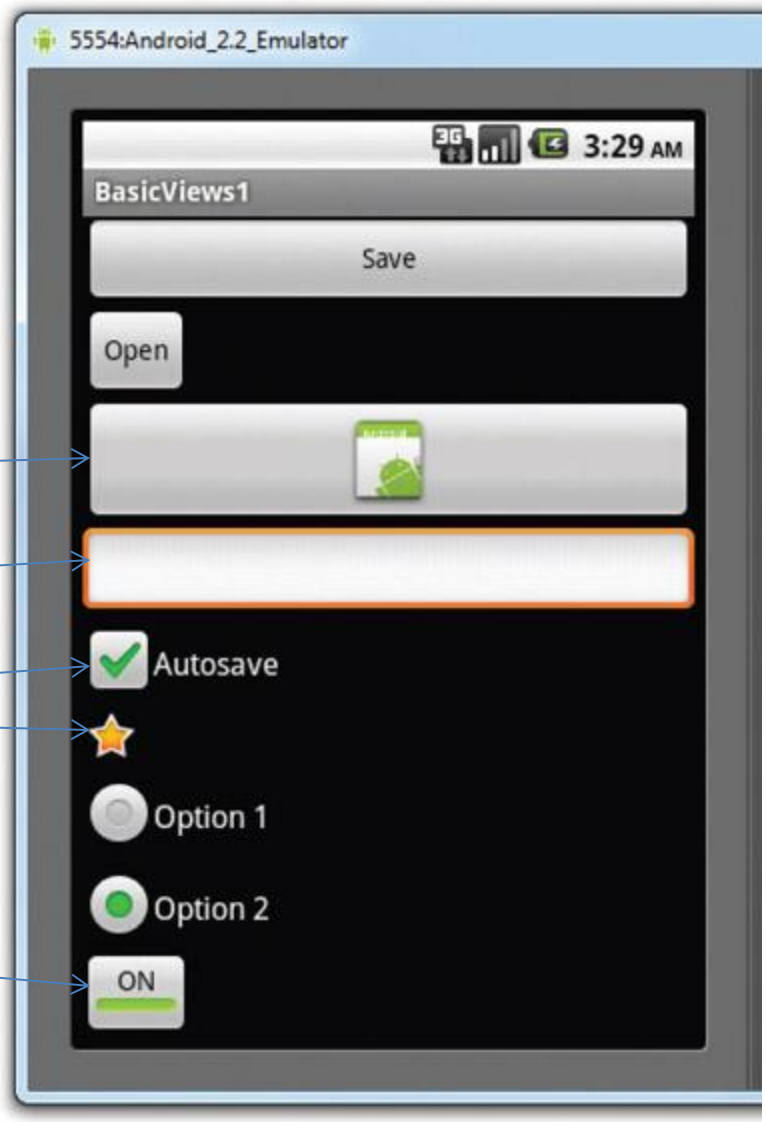
- Button
  - width = match_parent
  - width = wrap_content



- fill_parent / match_parent (>API 8): Setting the layout of a widget to fill_parent / match_parent will force it to expand to take up as much space as is available within the layout element it's been placed in.
- wrap_content: Setting a view‘s size to wrap_content will force it to expand only far enough to contain the values (or child controls) it contains.

Other Views

- ImageButton
- EditText
- CheckBox
- RadioGroup
- ToggleButton





Android Layout

Layout Specification

- XML file – structured, human readable doc format, like html
  - Static
    - e.g., Hello World app
  - More orthogonal code
    - e.g., different display size, orientation, language
  - Easier to work with
    - Visualization, debug
  - Common in most situations
- Can support run-time modifications
  - e.g., XML for default/initial layout, and modify at run-time
- For Android
  - XML element/attribute parallels to Android class/method
    - <TextView> element – TextView widget
    - <LinearLayout> element – LinearLayout view-group

XML Layout Example

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
        android:layout_width="fill_parent"
        android:layout_height="fill_parent"
        android:orientation="vertical" >
   <TextView android:id="@+id/text"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, I am a TextView" />
   <Button android:id="@+id/button"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Hello, I am a Button" />
</LinearLayout>
```

XML Layout Example

- At run time
  - inflate the xml for Activity.onCreate()
  - no additional coding!

- ```
  public void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_main);
  }
  ```
  ```

XML Attributes

- ID
  - Needed if the view needs to be referenced by other views or Java / Kotlin code
  - e.g., under layout xml
    - android:id="@+id/my_button“
  - @: parse and expand
  - +: add to resource (i.e., R.java)
  - Under onCreate()
    - Button myButton = (Button) **findViewById**(**R.id.**my_button);
- Layout parameters
  - layout_*, e.g.
    - <Button android:id="@+id/my_button"
      android:layout_width=“fill_parent"
      android:layout_height="wrap_content"
      android:text="@string/my_button_text"/>
  - View class dependent

Common Layout

- FrameLayout
- LinearLayout
- RelativeLayout
- TableLayout
- ConstraintLayout

Although Constraint Layout is the default and the recommended layout, we will still introduce other layouts because:

1) **Lots of existing projects are still adopting other layouts.**
2) **Some layouts (e.g., Linear Layout) are easier to understand than Constraint Layout.**

FrameLayout

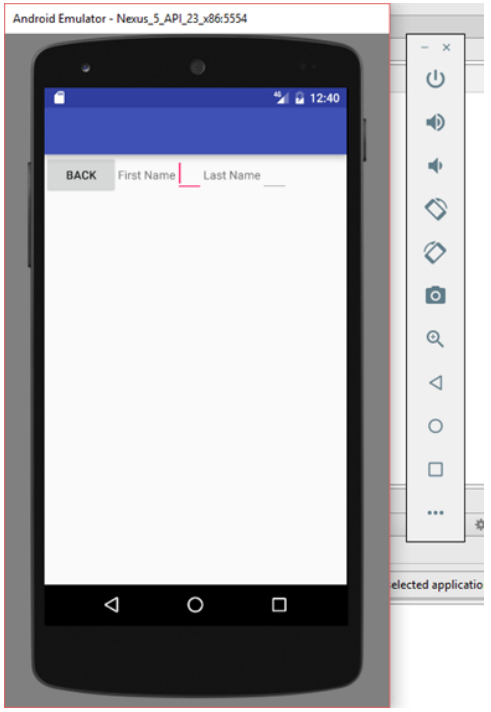
- Display a single item at a time
- Elements are displayed overlapping

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   xmlns:android="http://schemas.android.com/apk/res/android">
   <ImageView
      android:src="@drawable/hkulogo"
      android:scaleType="fitCenter"
      android:layout_height="match_parent"
      android:layout_width="match_parent"/>
   <TextView
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:textSize="24sp"
      android:textColor="#000000"
      android:gravity="center"
      android:text="I love HKU!\n I love Computer Science!\n I love mobile app development!"/>
</FrameLayout>
```



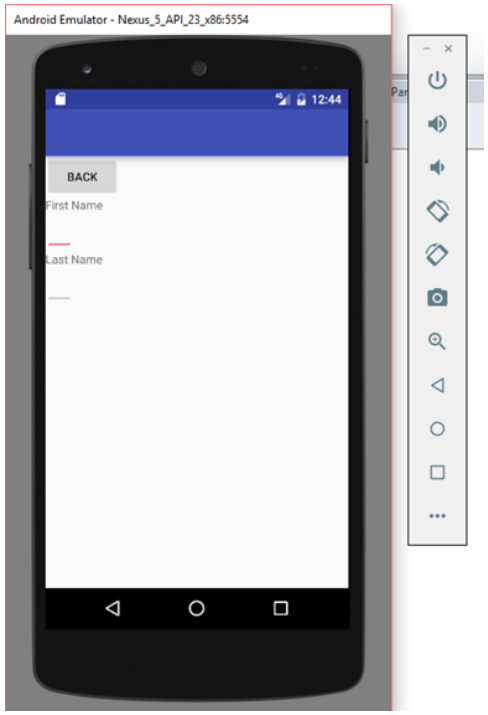
LinearLayout

- Organize elements along a single line
- Can be horizontal or vertical (by setting android:orientation)




```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context="hkucs.test3.MainActivity">
    <Button
        android:id="@+id/backbutton"
        android:text="Back"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />
    <TextView
        android:text="First Name"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />
    <EditText
        android:width="100px"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />
    <TextView
        android:text="Last Name"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />
    <EditText
        android:width="100px"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />
</LinearLayout>
```

LinearLayout




```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
   xmlns:tools="http://schemas.android.com/tools"
   android:orientation="vertical"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   tools:context="hkucs.test3.MainActivity">
   <Button
      android:id="@+id/backbutton"
      android:text="Back"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content" />
   <TextView
      android:text="First Name"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content" />
      <EditText
         android:width="100px"
         android:layout_width="wrap_content"
         android:layout_height="wrap_content" />
      <TextView
         android:text="Last Name"
         android:layout_width="wrap_content"
         android:layout_height="wrap_content" />
      <EditText
         android:width="100px"
         android:layout_width="wrap_content"
         android:layout_height="wrap_content" />
</LinearLayout>
```

RelativeLayout

- Lay out elements based on their relationships with one another, and with the parent container




```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
xmlns:android="http://schemas.android.com/apk
/res/android"
xmlns:tools="http://schemas.android.com/tools"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   tools:context="hkucs.test5.MainActivity">
   <Button
      android:id="@+id/backbutton"
      android:text="Back"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content" />
   <TextView
      android:id="@+id/firstName"
      android:text="First Name"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_below="@id/backbutton"/>
   <EditText
      android:id="@+id/editFirstName"
      android:width="500px"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_toRightOf="@id/firstName"
      android:layout_below="@id/backbutton"/>
   <EditText
      android:id="@+id/editLastName"
      android:width="500px"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_below="@id/editFirstName"
      android:layout_alignLeft="@id/editFirstName"/>
   <TextView
      android:id="@+id/lastName"
      android:text="Last Name"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:layout_toLeftOf="@id/editLastName"
      android:layout_below="@id/editFirstName"/>
</RelativeLayout>
```

RelativeLayout

- These properties will layout elements relative to the parent container.
  - android:layout_alignParentBottom – Places the bottom of the element on the bottom of the container
  - android:layout_alignParentLeft – Places the left of the element on the left side of the container
  - android:layout_alignParentRight – Places the right of the element on the right side of the container
  - android:layout_alignParentTop – Places the element at the top of the container
- These properties will layout elements relative to the parent container.
  - android:layout_centerHorizontal – Centers the element horizontally within its parent container
  - android:layout_centerInParent – Centers the element both horizontally and vertically within its container
  - android:layout_centerVertical – Centers the element vertically within its parent container

RelativeLayout

- These properties allow you to specify how elements are aligned in relation to other elements.
  - android:layout_above – Places the element above the specified element
  - android:layout_below – Places the element below the specified element
  - android:layout_toLeftOf – Places the element to the left of the specified element
  - android:layout_toRightOf – Places the element to the right of the specified element
- android:layout_alignBaseline – Aligns baseline of the new element with the baseline of the specified element
- android:layout_alignBottom – Aligns the bottom of new element in with the bottom of the specified element
- android:layout_alignLeft – Aligns left edge of the new element with the left edge of the specified element
- android:layout_alignRight – Aligns right edge of the new element with the right edge of the specified element
- android:layout_alignTop – Places top of the new element in alignment with the top of the specified element

TableLayout

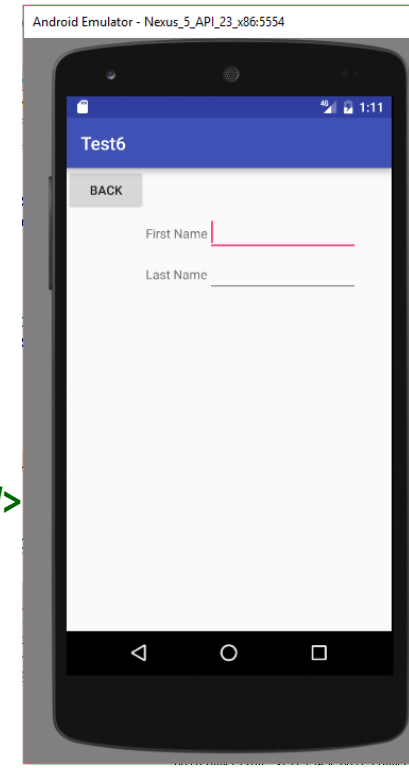
- TableLayout organizes content into rows and columns. The rows are defined in the layout XML, and the columns are determined automatically by Android. This is done by creating at least one column for each element. So, for example, if you had a row with two elements and a row with five elements then you would have a layout with two rows and five columns.
- You can specify that an element should occupy more than one column using android:layout_span. This can increase the total column count as well, so if we have a row with two elements and each element has android:layout_span=”3” then you will have at least six columns in your table.
- By default, Android places each element in the first unused column in the row. You can, however, specify the column an element should occupy using android:layout_column.

TableLayout

```
<?xml version="1.0" encoding="utf-8"?>
<TableLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   tools:context="hkucs.test6.MainActivity">
   <TableRow>
      <Button
         android:id="@+id/backbutton"
         android:text="Back"
         android:layout_width="wrap_content"
         android:layout_height="wrap_content" />
   </TableRow>
```

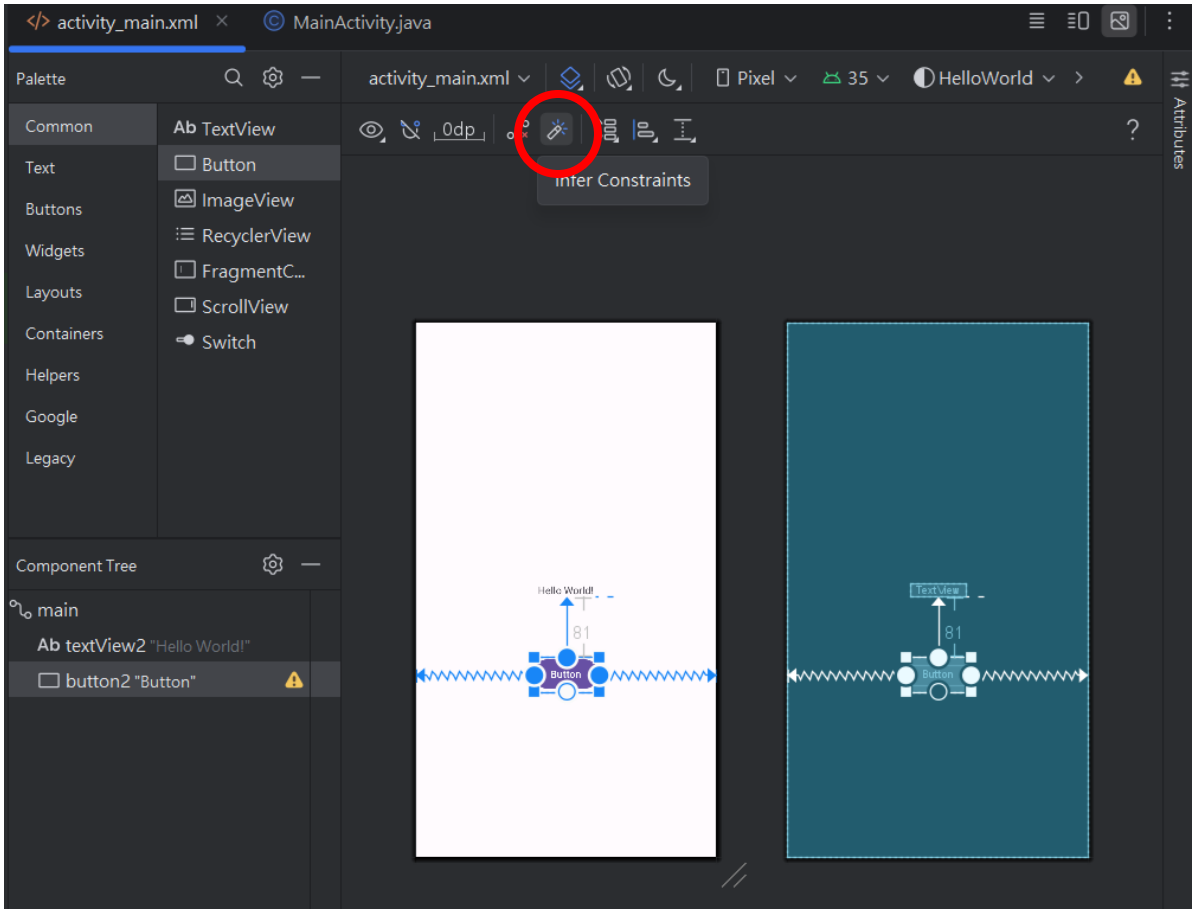
Challenge:
How many rows and columns are there in this table?
Answer:
3 rows and 3 columns

```
   <TableRow>
      <TextView
         android:text="First Name"
         android:layout_width="wrap_content"
         android:layout_height="wrap_content"
         android:layout_column="1"/>
      <EditText
         android:width="500px"
         android:layout_width="wrap_content"
         android:layout_height="wrap_content"/>
   </TableRow>
   <TableRow>
      <TextView
         android:text="Last Name"
         android:layout_width="wrap_content"
         android:layout_height="wrap_content"
         android:layout_column="1"/>
      <EditText
         android:width="500px"
         android:layout_width="wrap_content"
         android:layout_height="wrap_content"/>
   </TableRow>
</TableLayout>
```



ConstraintLayout

- Latest addition to Android's collection of layouts. Default layout in Android Studio 2.2 or above.
- You can drag the components you want one by one.
- Select the components, click the circled button ("Infer Constraints") to define their positions relative to the parent frame or to other components.



ConstraintLayout

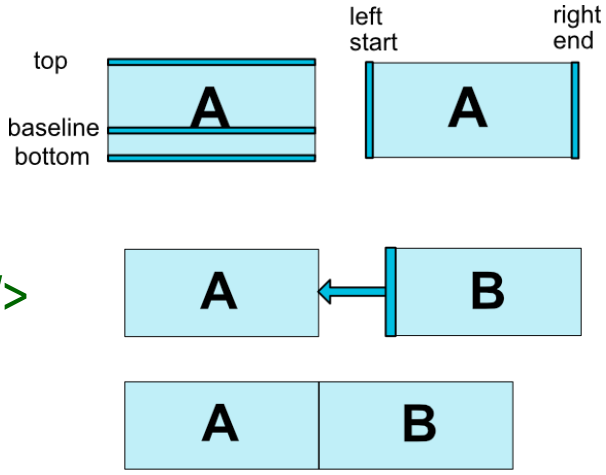
```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
    <TextView
        android:id="@+id/textView2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
    <Button
        android:id="@+id/button2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="81dp"
        android:text="Button"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/textView2" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

ConstraintLayout

- Relative positioning is one of the basic building block of creating layouts in ConstraintLayout. Those constraints allow you to position a given widget relative to another one.
- You can constrain a component on the horizontal and vertical axis:
  - Horizontal Axis: Left / Start, Right / End sides
  - Vertical Axis: top, bottom sides and text baseline

- Example:

```
<Button android:id="@+id/buttonA" ... />
<Button android:id="@+id/buttonB" ...
app:layout_constraintLeft_toRightOf="@+id/buttonA" />
```



ConstraintLayout

- List of available constraints:
  - layout_constraintLeft_toLeftOf
  - layout_constraintLeft_toRightOf
  - layout_constraintRight_toLeftOf
  - layout_constraintRight_toRightOf
  - layout_constraintTop_toTopOf
  - layout_constraintTop_toBottomOf
  - layout_constraintBottom_toTopOf
  - layout_constraintBottom_toBottomOf
  - layout_constraintBaseline_toBaselineOf
  - layout_constraintStart_toEndOf
  - layout_constraintStart_toStartOf
  - layout_constraintEnd_toStartOf
  - layout_constraintEnd_toEndOf

Think about when we will use layout_constraintLeft_toLeftOf and layout_constraintTop_toTopOf?

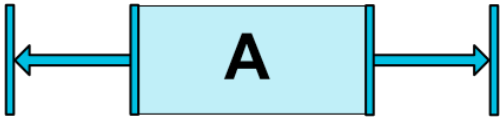
ConstraintLayout

- You can also set a margin between objects:
  - android:layout_marginStart
  - android:layout_marginEnd
  - android:layout_marginLeft
  - android:layout_marginTop
  - android:layout_marginRight
  - android:layout_marginBottom
- Note that a margin can only be positive or equals to zero, and takes a Dimension.
- In our previous example, we use margin to set a component to a certain position:
  - android:layout_marginStart="150dp“
  - android:layout_marginTop="64dp"

ConstraintLayout

- More examples:

```
<android.support.constraint.ConstraintLayout ...>
        <Button android:id="@+id/button" ...
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent/>
</>
```



Contradiction???

```
<android.support.constraint.ConstraintLayout ...>
        <Button android:id="@+id/button" ...
        app:layout_constraintHorizontal_bias="0.3"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent/>
</>
```



ConstraintLayout

- You can also define one dimension of a widget as a ratio of the other one. In order to do that, you need to have at least one constrained dimension be set to 0dp (i.e., MATCH_CONSTRAINT), and set the attribute layout_constraintDimentionRatio to a given ratio.
- Example:

```
<Button   android:layout_width="wrap_content"
          android:layout_height="0dp"
          app:layout_constraintDimensionRatio="1:1" />
```

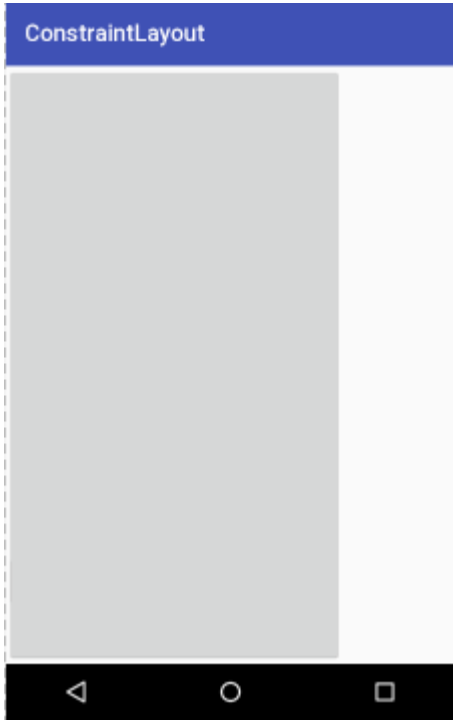
This will set the height of the button to be the same as its width.

- The ratio can be expressed either as:
  - a float value, representing a ratio between width and height
  - a ratio in the form "width:height"

ConstraintLayout

- Another example:

```
<Button android:layout_width="0dp"
        android:layout_height="0dp"
        app:layout_constraintDimensionRatio="H,16:9"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintTop_toTopOf="parent"/>
```



This will set the height of the button following a 16:9 ratio, while the width of the button will match the constraints to parent.

Please refer to the following site for more information:
https://developer.android.com/reference/android/support/constraint/ConstraintLayout.html

ScrollView

- Layout container for a view hierarchy that can be scrolled by the user, allowing it to be larger than the physical display.
- A ScrollView is a FrameLayout, meaning you should place one child in it containing the entire contents to scroll. This child may itself be a layout manager with a complex hierarchy of objects.

- Example:

```
<ScrollView
xmlns:android=http://schemas.android.com/apk/res/android
android:layout_width="fill_parent“
android:layout_height="fill_parent“
android:fillViewport="true">
        <RelativeLayout
        android:layout_width="fill_parent"
        android:layout_height="wrap_content“
        android:background="#ffffff">
        …
        </RelativeLayout>
</ScrollView>
```





Android Dimension Issues

Example

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
   xmlns:tools="http://schemas.android.com/tools"
   android:orientation="horizontal"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   tools:context="hkucs.test.MainActivity">
   <Button
      android:id="@+id/backbutton"
      android:text="Back"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content" />
   <TextView
      android:text="First Name"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content" />
   <EditText
      android:width="100px"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content" />
   <TextView
      android:text="Last Name"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content" />
   <EditText
      android:width="100px"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content" />
</LinearLayout>
```

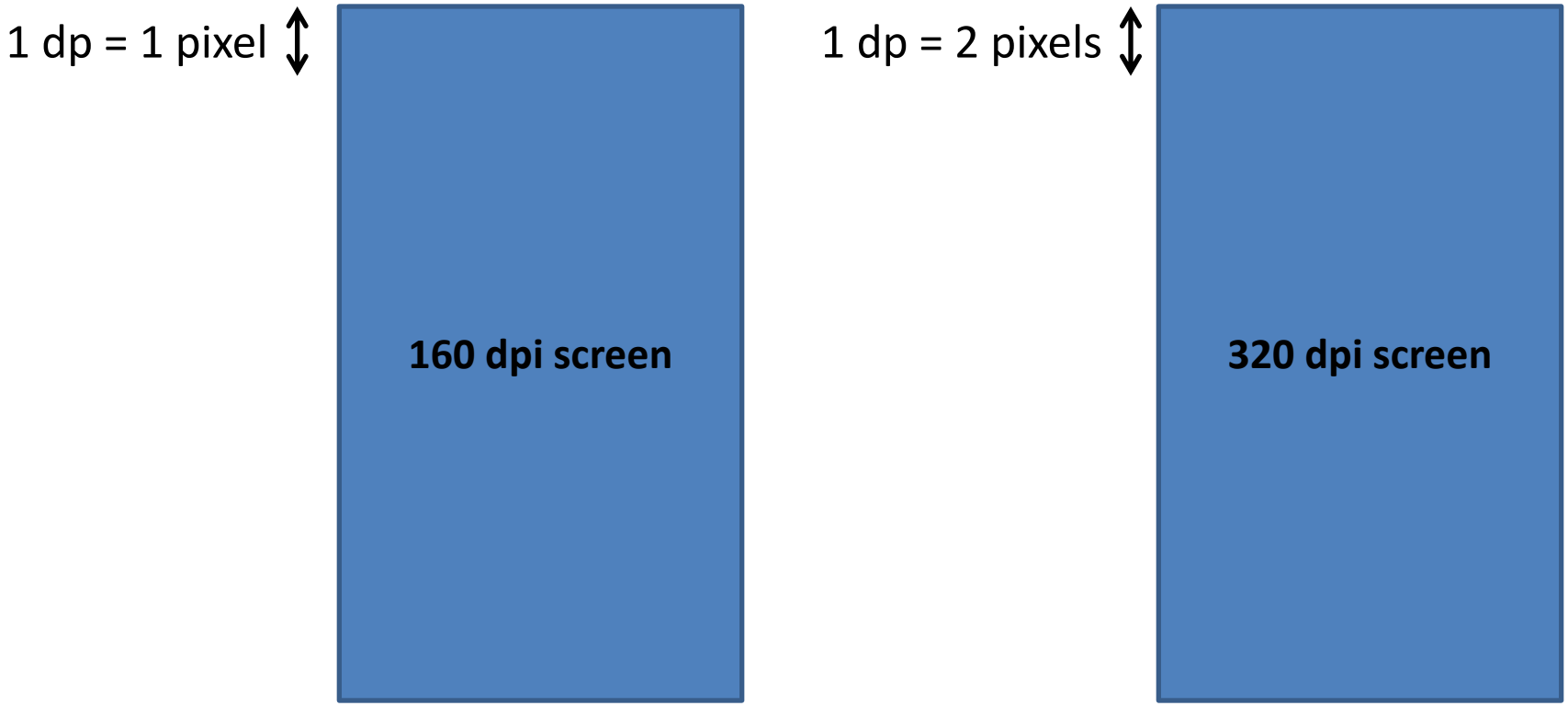
px, in, mm, pt

- px = Pixels
  - Corresponds to actual pixels on the screen.
- in = Inches
  - Based on the physical size of the screen.
- mm = Millimeters
  - Based on the physical size of the screen.
- pt = Points
  - 1/72 of an inch based on the physical size of the screen.

dp, sp

- dp = Density-independent Pixels
  - An abstract unit that is based on the physical density of the screen. These units are relative to a 160 dpi screen, so one dp is one pixel on a 160 dpi screen. The ratio of dp-to-pixel will change with the screen density, but not necessarily in direct proportion.
  - Note: The compiler accepts both "dip" and "dp", though "dp" is more consistent with "sp".
- sp = Scale-independent Pixels
  - This is like the dp unit, but it is also scaled by the user's font size preference. It is recommended you use this unit when specifying font sizes, so they will be adjusted for both the screen density and user's preference.

dp



sp

- Setting text size:
  - Define the following in an XML file such as dimensions.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
  <dimen name="textsize">15sp</dimen>
</resources>
```

  - You can then set the text size in Java using:

```
textView.setTextSize(getResources().getDimension(R.dimen.textsize));
```

Chapter 4.

SCHOOL OF COMPUTING & DATA SCIENCE
The University of Hong Kong

End

